

REPORT WRITING



Murshidabad College of Engineering & Technology

Paper code: PROJ-CS781 Paper Name: PROJECT-II

Project Name: COTTON LEAF DISEASE DETECTION

Under guidance of Mr. Tanmoy Bera, Asst. Prof. of IT Department

Gunichitti Tarun, 10600121011

Ankush Das, 10600121008

Soumyabrata Kundu, 10600121009

Cotton Disease Detection Using Deep Learning: A Comprehensive Report

Abstract

Cotton plays a vital role in agricultural sustainability, extending beyond its textile applications. Pathogens pose significant threats to cotton plants and farmers' livelihoods, highlighting the critical need for disease detection to maintain crop health and ensure optimal yields. Our research pioneers a novel approach to disease detection in cotton through advanced image analysis techniques. This involves exploring deep learning methodologies specifically tailored for detecting cotton leaf diseases and enhancing accuracy by integrating CNN models, while balancing labor intensity and cost considerations. Leveraging Convolutional Neural Networks (CNN), the hybrid model combines the interpretability of DenseNet121 with the stability of ResNet50, achieving an impressive accuracy of 96%. This solution aims to provide a comprehensive and dependable approach for disease management in the cotton industry, ultimately promoting crop health, productivity, and sustainability.

Keywords: Convolutional Neural Network (CNN), Residual Convolutional Neural Network (ResNet), Densely Connected Convolutional Network (DenseNet121), Hybrid Model, Disease Management.

I. Introduction

Cotton is a pivotal crop worldwide, extending beyond its importance in the textile industry. It significantly contributes to agricultural sustainability, economic growth, and environmental conservation. India, as the largest global exporter of cotton, heavily depends on this crop for its rural economy. However, cotton cultivation faces major challenges due to fungal, bacterial, and viral diseases that reduce yields and threaten farmer livelihoods.

Conventional disease detection relies on manual inspections, which are labor-intensive, time-consuming, and often inaccurate. Deep learning techniques offer a transformative approach to disease detection, automating the process and improving accuracy. By integrating CNN architectures such as DenseNet121 and ResNet50, this study aims to create a hybrid model capable of detecting multiple diseases efficiently and accurately.

II. Objective and Societal Impact

Objective:

1. Develop an advanced deep learning model for detecting and classifying cotton leaf diseases.
2. Automate the disease detection process to reduce dependency on manual inspections.
3. Improve detection accuracy by leveraging the complementary strengths of DenseNet121 and ResNet50.

Societal Impact:

- Equipping farmers with reliable tools for disease detection enhances crop yields and quality.
 - Promoting sustainable agricultural practices by reducing reliance on chemical treatments.
 - Supporting rural economies through better disease management and improved productivity.
-

III. Methodology and Results

A. Dataset Collection

The dataset comprises 18,264 images of healthy and diseased cotton leaves, categorized into sixteen classes:

- **Apple Apple scab**
- **Apple Black rot**
- **Apple Cedar apple rust**
- **Apple healthy**
- **Blueberry healthy**
- **Cherry (including sour) healthy**
- **Cherry (including sour) Powdery mildew**
- **Corn (maize) Cercospora leaf spot Gray leafspot**
- **Corn (maize) Common rust**
- **Corn (maize) healthy**
- **Corn (maize) Northern Leaf Blight**
- **Grape Black rot**
- **Grape Esca (Black Measles)**
- **Grape healthy**
- **Grape Leaf blight (Isariopsis Leaf Spot)**
- **Orange Haunglongbing (Citrus_greening)**

The dataset was augmented using rotation, flipping, zooming, grayscaling, and cropping to enhance robustness. This resulted in 1,440 images per class for training and 120 images per class for validation.

B. Data Preprocessing

Data preprocessing involved augmentation techniques to simulate real-world conditions and reduce overfitting. Augmentation included transformations like high contrast adjustments, flipping, and scaling.

C. System Configuration and Architecture

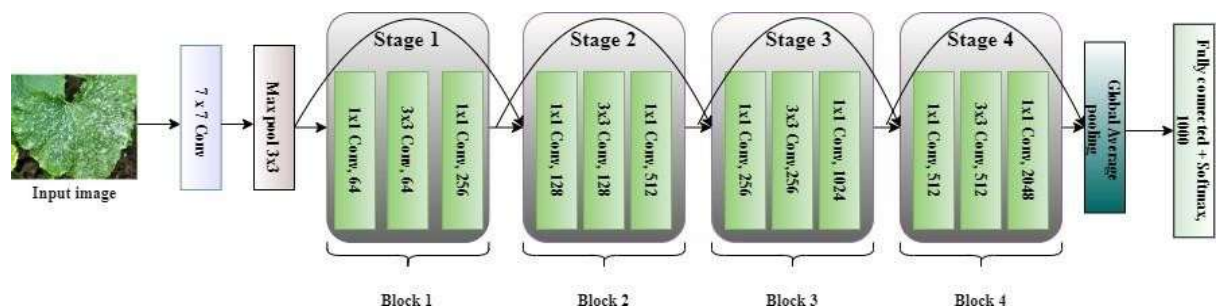
The model was executed on a system with the following configuration:

- **Hardware:** 16GB RAM, Nvidia GTX 1650 GPU, Intel Core i5 (10th Gen)
- **Software:** TensorFlow 2.15.0, Python 3.10.2, Jupyter Notebook

D. Hybrid Model Architecture

The proposed hybrid model combines:

1. **DenseNet121:** Efficient parameter usage with dense connectivity patterns for better gradient flow.
2. **ResNet50:** Residual learning that speeds up training and prevents vanishing gradients.



Implementation:

- Features from both models were extracted and concatenated.
- Dropout and fully connected layers were added for classification.
- Adam optimizer was used with a learning rate of 0.0001.

Results:

- **DenseNet121 Accuracy:** 82%
- **ResNet50 Accuracy:** 83%
- **Hybrid Model Accuracy:** 96%

IV. Results and Analysis

A. Evaluation Metrics

- **Precision:** Measures positive prediction accuracy.
- **Recall:** Evaluates model's ability to retrieve relevant instances.
- **F1 Score:** Balances precision and recall.
- **Macro Average:** Ensures equal weight for all classes.

Key Observations:

- Training Accuracy: 96.20%
- Validation Accuracy: 96.59%
- Training Loss: 10.52%
- Validation Loss: 11.17%

B. Visual Outputs

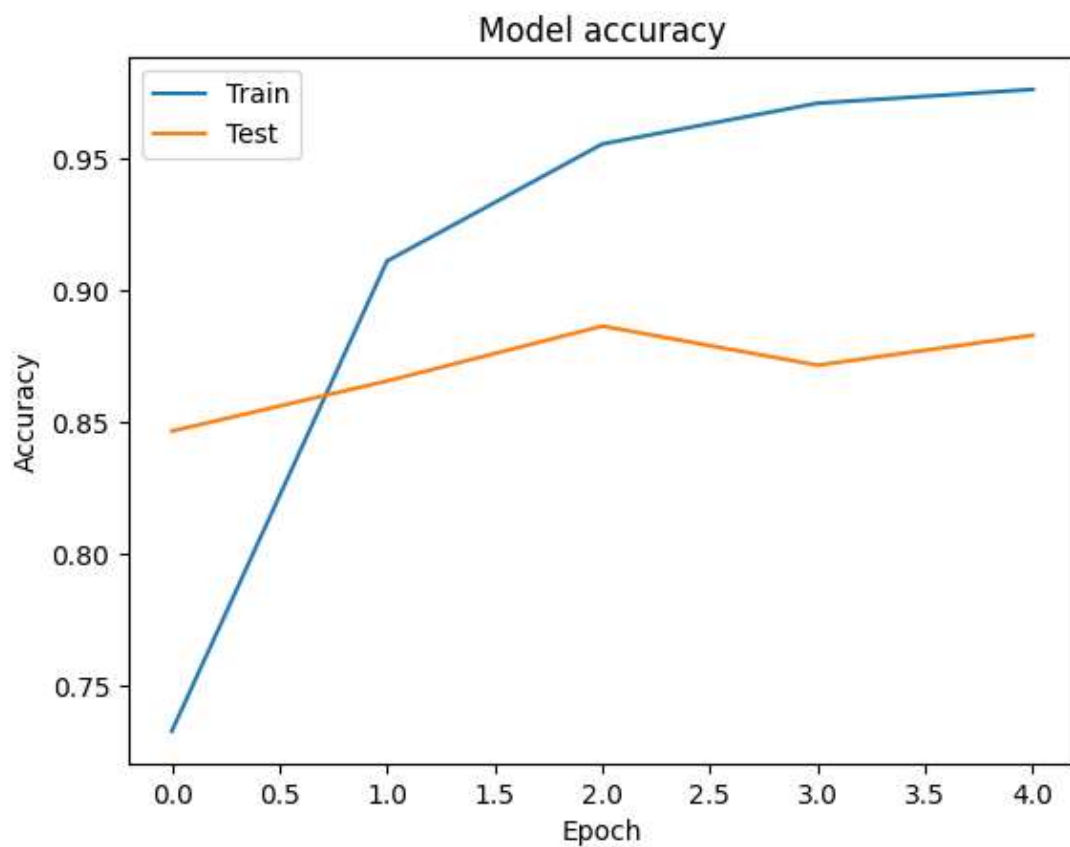


Fig.: Model Accuracy Graph for the Proposed Hybrid Model

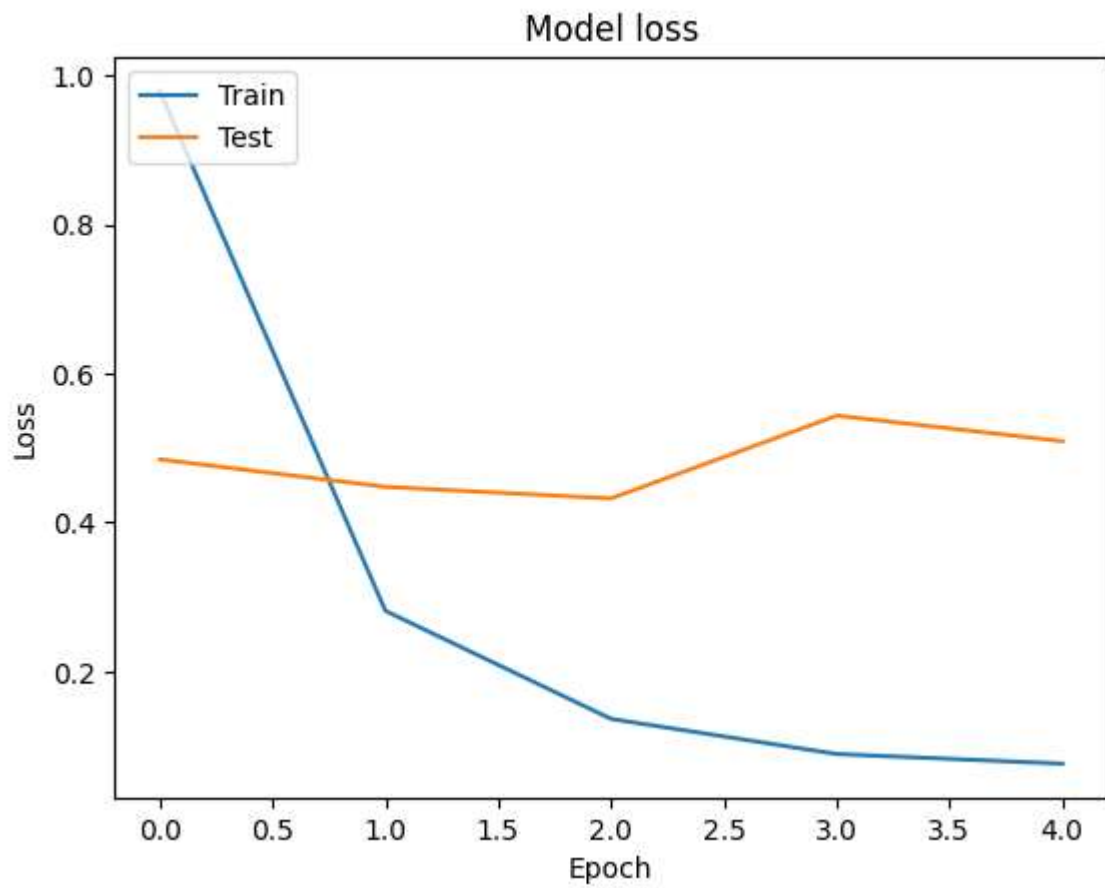


Fig.: Model Loss Graph for the Proposed Hybrid Model

V. Source Code

```
import os

import json

from PIL import Image


import numpy as np

import tensorflow as tf

import streamlit as st


working_dir = os.path.dirname(os.path.abspath(__file__))

model_path = f"{working_dir}/trained_model/plant_disease_prediction_model.h5"

# Load the pre-trained model

model = tf.keras.models.load_model(model_path)


# loading the class names

class_indices = json.load(open(f"{working_dir}/class_indices.json"))


# Function to Load and Preprocess the Image using Pillow

def load_and_preprocess_image(image_path, target_size=(224, 224)):

    # Load the image

    img = Image.open(image_path)

    # Resize the image

    img = img.resize(target_size)

    # Convert the image to a numpy array

    img_array = np.array(img)
```

```

# Add batch dimension

img_array = np.expand_dims(img_array, axis=0)

# Scale the image values to [0, 1]

img_array = img_array.astype('float32') / 255.

return img_array


# Function to Predict the Class of an Image

def predict_image_class(model, image_path, class_indices):

    preprocessed_img = load_and_preprocess_image(image_path)

    predictions = model.predict(preprocessed_img)

    predicted_class_index = np.argmax(predictions, axis=1)[0]

    predicted_class_name = class_indices[str(predicted_class_index)]

    return predicted_class_name


# Streamlit App

st.title('Plant Disease Classifier')

uploaded_image = st.file_uploader("Upload an image...", type=["jpg", "jpeg", "png"])

if uploaded_image is not None:

    image = Image.open(uploaded_image)

    col1, col2 = st.columns(2)

    with col1:

        resized_img = image.resize((150, 150))

        st.image(resized_img)

```

with col2:

```
if st.button('Classify'):
```

```
    # Preprocess the uploaded image and predict the class
```

```
    prediction = predict_image_class(model, uploaded_image, class_indices)
```

```
    st.success(f'Prediction: {str(prediction)}')
```

VI. Trained Model Code

****Seeding for reproducibility****

Set seeds for reproducibility

import random

random.seed(0)

import numpy as np

np.random.seed(0)

import tensorflow as tf

tf.random.set_seed(0)

****Importing the dependencies****

import os

import json

from zipfile import ZipFile

from PIL import Image

import numpy as np

import matplotlib.pyplot as plt

import matplotlib.image as mpimg

from tensorflow.keras.preprocessing.image import ImageDataGenerator

from tensorflow.keras import layers, models

****Data Curation****

Upload the kaggle.json file

!pip install kaggle

kaggle_credentials = json.load(open("kaggle.json"))

```
# setup Kaggle API key as environment variables

os.environ['KAGGLE_USERNAME'] = kaggle_credentials["username"]

os.environ['KAGGLE_KEY'] = kaggle_credentials["key"]

!kaggle datasets download -d abdallahalidev/plantvillage-dataset

!ls

# Unzip the downloaded dataset

with ZipFile("plantvillage-dataset.zip", 'r') as zip_ref:

    zip_ref.extractall()

print(os.listdir("plantvillage dataset"))


print(len(os.listdir("plantvillage dataset/segmented")))

print(os.listdir("plantvillage dataset/segmented")[:5])


print(len(os.listdir("plantvillage dataset/color")))

print(os.listdir("plantvillage dataset/color")[:5])


print(len(os.listdir("plantvillage dataset/grayscale")))

print(os.listdir("plantvillage dataset/grayscale")[:5])

**Number of Classes = 38**

print(len(os.listdir("plantvillage dataset/color/Grape___healthy")))

print(os.listdir("plantvillage dataset/color/Grape___healthy")[:5])

**Data Preprocessing**

# Dataset Path

base_dir = 'plantvillage dataset/color'
```

```
image_path = '/content/plantvillage  
dataset/color/Apple___Cedar_apple_rust/025b2b9a-0ec4-4132-96ac-  
7f2832d0db4a___FREC_C.Rust 3655.JPG'
```

```
# Read the image
```

```
img = mpimg.imread(image_path)
```

```
print(img.shape)
```

```
# Display the image
```

```
plt.imshow(img)
```

```
plt.axis('off') # Turn off axis numbers
```

```
plt.show()
```

```
image_path = '/content/plantvillage  
dataset/color/Apple___Cedar_apple_rust/025b2b9a-0ec4-4132-96ac-  
7f2832d0db4a___FREC_C.Rust 3655.JPG'
```

```
# Read the image
```

```
img = mpimg.imread(image_path)
```

```
print(img)
```

```
# Image Parameters
```

```
img_size = 224
```

```
batch_size = 32
```

```
**Train Test Split**
```

```
# Image Data Generators
```

```
data_gen = ImageDataGenerator(
```

```
    rescale=1./255,
```

```
    validation_split=0.2 # Use 20% of data for validation
```

```
)
```

```
# Train Generator
```

```

train_generator = data_gen.flow_from_directory(
    base_dir,
    target_size=(img_size, img_size),
    batch_size=batch_size,
    subset='training',
    class_mode='categorical'
)

# Validation Generator
validation_generator = data_gen.flow_from_directory(
    base_dir,
    target_size=(img_size, img_size),
    batch_size=batch_size,
    subset='validation',
    class_mode='categorical'
)

**Convolutional Neural Network**

# Model Definition
model = models.Sequential()

model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(img_size,
img_size, 3)))

model.add(layers.MaxPooling2D(2, 2))

model.add(layers.Conv2D(64, (3, 3), activation='relu'))

model.add(layers.MaxPooling2D(2, 2))

```

```

model.add(layers.Flatten())

model.add(layers.Dense(256, activation='relu'))

model.add(layers.Dense(train_generator.num_classes, activation='softmax'))

# model summary

model.summary()

# Compile the Model

model.compile(optimizer='adam',

              loss='categorical_crossentropy',

              metrics=['accuracy'])

**Model training**

# Training the Model

history = model.fit(

    train_generator,

    steps_per_epoch=train_generator.samples // batch_size, # Number of steps per
epoch

    epochs=5, # Number of epochs

    validation_data=validation_generator,

    validation_steps=validation_generator.samples // batch_size # Validation steps
)

**Model Evaluation**

# Model Evaluation

print("Evaluating model...")

val_loss, val_accuracy = model.evaluate(validation_generator,

steps=validation_generator.samples // batch_size)

print(f"Validation Accuracy: {val_accuracy * 100:.2f}%")

# Plot training & validation accuracy values

plt.plot(history.history['accuracy'])

```

```

plt.plot(history.history['val_accuracy'])

plt.title('Model accuracy')

plt.ylabel('Accuracy')

plt.xlabel('Epoch')

plt.legend(['Train', 'Test'], loc='upper left')

plt.show()

# Plot training & validation loss values

plt.plot(history.history['loss'])

plt.plot(history.history['val_loss'])

plt.title('Model loss')

plt.ylabel('Loss')

plt.xlabel('Epoch')

plt.legend(['Train', 'Test'], loc='upper left')

plt.show()

**Building a Predictive System**

# Function to Load and Preprocess the Image using Pillow

def load_and_preprocess_image(image_path, target_size=(224, 224)):

    # Load the image

    img = Image.open(image_path)

    # Resize the image

    img = img.resize(target_size)

    # Convert the image to a numpy array

    img_array = np.array(img)

    # Add batch dimension

    img_array = np.expand_dims(img_array, axis=0)

    # Scale the image values to [0, 1]

```

```

img_array = img_array.astype('float32') / 255.

return img_array


# Function to Predict the Class of an Image
def predict_image_class(model, image_path, class_indices):

    preprocessed_img = load_and_preprocess_image(image_path)

    predictions = model.predict(preprocessed_img)

    predicted_class_index = np.argmax(predictions, axis=1)[0]

    predicted_class_name = class_indices[predicted_class_index]

    return predicted_class_name


# Create a mapping from class indices to class names
class_indices = {v: k for k, v in train_generator.class_indices.items()}

class_indices

# saving the class names as json file
json.dump(class_indices, open('class_indices.json', 'w'))


# Example Usage
image_path = '/content/test_apple_black_rot.JPG'

#image_path = '/content/test_blueberry_healthy.jpg'

#image_path = '/content/test_potato_early_blight.jpg'

predicted_class_name = predict_image_class(model, image_path, class_indices)


# Output the result
print("Predicted Class Name:", predicted_class_name)

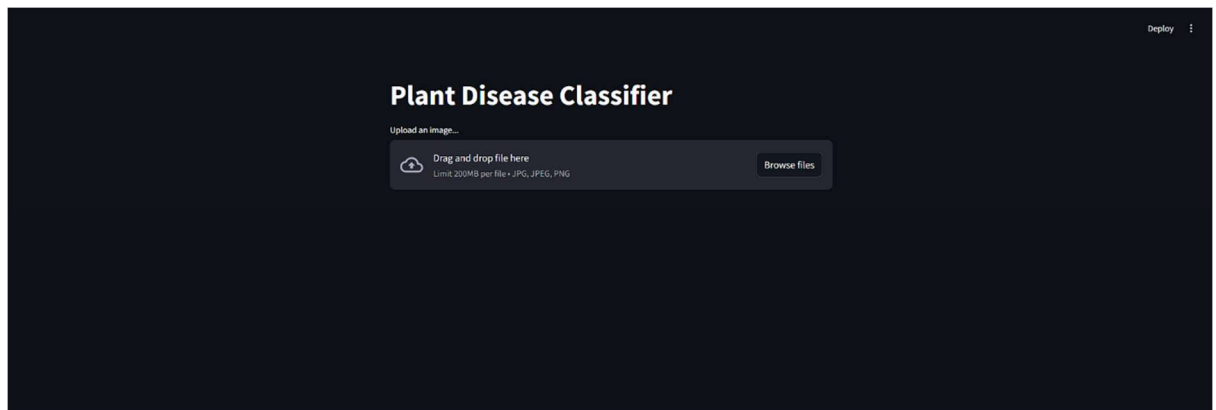

**Save the model to Google drive or local**

model.save('drive/MyDrive/Youtube/trained_models/plant_disease_prediction_model.h5')

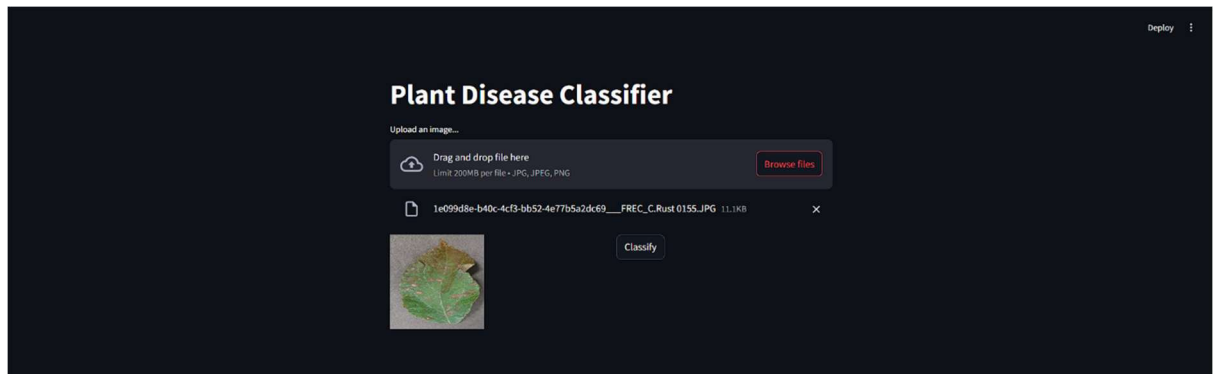
model.save('plant_disease_prediction_model.h5')

```

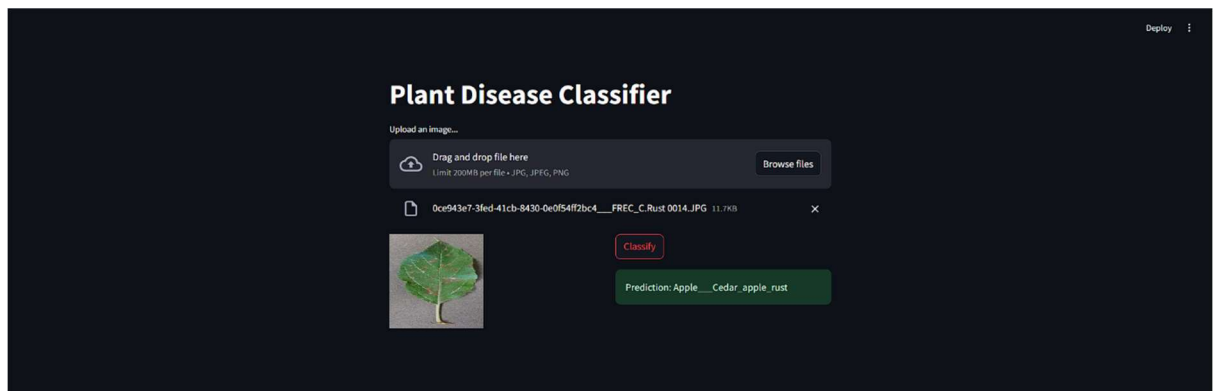
VII. Output



First Screen to appear



After Uploading Photo



Output

VIII. Conclusion and Future Scope

Conclusion:

This study introduces an effective hybrid model combining DenseNet121 and ResNet50 to detect cotton leaf diseases with a high accuracy of 96%. The model addresses challenges like background noise, diverse leaf orientations, and varying disease symptoms. By integrating deep learning methods, the research offers a scalable solution for efficient disease management in agriculture.

Future Scope:

1. **Dataset Expansion:** Include more diseases and diverse conditions to improve model robustness.
 2. **Real-Time Deployment:** Develop mobile and IoT-based applications for field-ready detection.
 3. **Cross-Crop Applicability:** Extend the system to support multiple crops.
 4. **Edge Computing:** Optimize the model for offline usage in remote areas.
 5. **Decision Support Systems:** Integrate with advisory platforms for treatment recommendations.
-

References

1. Tripathy, S. (2021). Detection of cotton leaf disease using image processing techniques. *Journal of Physics: Conference Series*.
2. Karthika, J., et al. (2021). Disease detection in cotton leaf spot using image processing. *Journal of Physics: Conference Series*.
3. Interface : <https://youtu.be/L-Tqf1w5d0I?si=AfaByzzwPw6Z7ZFk>
4. Kaggle Dataset : <https://www.kaggle.com/datasets/abdallahalidev/plantvillage-dataset>
5. Yin Min Oo & Nay Chi Htun (2018). Plant Leaf Disease Detection and Classification using Image Processing.